Escape the Network

Build a route planner for a weighted, undirected network. Implement BFS, DFS and Dijkstra yourself, then compare the routes and search behaviour on several networks.

Complete the prep HTML first. Allow 5–7 lessons of 90 minutes, or longer if needed. Finish all six stages; save a working version after each.

Stage 1 Represent locations and connections

Do this

1. Use the supplied A–F nodes and location names. Store an adjacency matrix or adjacency lists.

2. Starter edges: A–B 4, A–C 2, B–D 5, B–E 3, C–D 1, D–E 1, D–F 7, E–F 2. Draw and check them.

3. Store every undirected edge in both directions. Allow non-negative costs.

4. For a matrix, use consistent node order and an explicit absent-edge marker, such as None. Cost 0 is valid.

5. Write an alphabetical neighbour function. Validate labels, dimensions, symmetry and weights.

Check before moving on

A has neighbours B cost 4 and C cost 2. D has B 5, C 1, E 1 and F 7. All eight undirected edges appear in both directions, and absent edges are distinct from cost 0.

Watch out

An undirected edge needs both directions. Testing if cost is true will lose a legitimate zero-cost edge.

Stage 2 Accept a route query and temporary blockage

Do this

1. Ask for start and exit labels; reject unknown ones. Allow start = exit.

2. Allow one existing connection to be blocked, or no blockage. Validate both labels and the edge.

3. Copy the graph independently. Remove the blocked edge in both directions from the copy only.

4. Use that same query graph for all three algorithms. Preserve the original for later queries.

5. Agree results: route, cost, edge count and nodes expanded. Write a helper summing consecutive edge costs. No route needs an explicit message, not cost 0.

Check before moving on

Blocking D–E removes both directions only in the query copy. The next unblocked query still has D–E. Unknown labels and absent connections are rejected without corrupting the original.

Watch out

Copying only a matrix's outer list shares its rows. A list of visited nodes is not automatically a connected route.

Stage 3 Implement breadth first search

Do this

1. Put the start in a FIFO queue and mark it discovered.

2. Remove the oldest queued node. If it is the exit, stop; otherwise examine alphabetical neighbours.

3. For each undiscovered neighbour, mark it on enqueue and record its predecessor: the node that discovered it.

4. If the queue empties, report no route. Discovery tracking must prevent endless cycles.

5. Follow predecessors from exit to start, then reverse to get the route.

6. Count expansion when examining neighbours. Check the exit first so it is not expanded. Return route, edges, cost and expansions.

Check before moving on

On the starter A-to-F query, alphabetical BFS returns A–B–D–F: 3 edges, cost 16 and 5 expansions under the stated counting rule. A–A gives 0 expansions.

Watch out

BFS minimises edges, not total travel cost when weights differ. Marking discovery too late can enqueue a node more than once.

Stage 4 Implement depth first search

Do this

1. Use recursion or a LIFO stack to explore one branch before alternatives. State how alphabetical neighbour order is achieved.

2. Mark nodes visited on entry and skip visited neighbours to prevent cycles.

3. Stop when the exit is reached. If a branch fails, backtrack to try another unvisited neighbour.

4. Record predecessors and reconstruct a connected route. Explored dead ends need not be on that route.

5. Use the same result fields and expansion rule as BFS. Compare both searches on the starter and a dead-end graph.

Check before moving on

Recursive alphabetical DFS on the starter A-to-F query returns A–B–D–E–F, cost 12 and 5 expansions. It explores C but C is not on the returned route. Cycles terminate.

Watch out

DFS's first route is not guaranteed to have the fewest edges or lowest cost. Its visited order may contain dead ends that are absent from the route.

Stage 5 Implement Dijkstra for the cheapest route

Do this

1. Start distances: 0 for the start, infinity for others. Store predecessors and settled nodes.

2. Select the unsettled node with the smallest finite total distance. Use a hand-written linear scan and alphabetical tie rule.

3. Settle it. Stop if it is the exit; report no route if no finite unsettled node remains.

4. For each unsettled neighbour, calculate current distance + edge cost. If cheaper, update its distance and predecessor: this is relaxation.

5. Reconstruct the cheapest route. Use the common results and expansion rule. Reject negative weights; do not use a library shortest-path function.

6. Trace tentative distances by hand and compare with the program.

Check before moving on

The starter A-to-F minimum-cost route is A–C–D–E–F, cost 6. With D–E blocked, the optimum is A–B–E–F, cost 9. A zero-cost A–B edge gives optimum cost 5.

Watch out

Choose the smallest tentative total from the start, not the smallest next edge. Discovering the exit does not mean its cost is final; settle it before stopping.

Stage 6 Compare networks and submit evidence

Do this

1. Compare all three algorithms on the starter, a dead-end graph, an unreachable-exit graph and a direct A–F edge costing 20.

2. Dead-end test: add B–G 3 and G–H 2. Separate unreachable test: isolated G.

3. Record case, start, exit, blockage, algorithm, route, edges, cost and expansions. Apply one expansion rule consistently.

4. Test start = exit, cycles, zero costs, negative rejection, unknown labels and original-graph preservation.

5. Explain different objectives and routes. State your graph representation when quoting complexity.

6. Submit code, README, comparison, findings, expected/actual tests and a 200–300 word reflection: actual bug, fix, two tests, limitation and help used.

Check before moving on

Every route uses real unblocked edges; costs add correctly; unreachable exits report no route. The comparison demonstrates BFS's edge objective, DFS's order dependence and Dijkstra's cost objective.

Watch out

Equal expansion counts do not imply identical total work. O(V+E) for BFS/DFS assumes adjacency lists; scanning a matrix changes the bound.